

Step 1 : Import Required Libraries

```
In [1]: import pandas as pd
import duckdb
```

Step 2 : Load the Dataset

```
In [2]: df = pd.read_csv("Sample - Superstore.csv", encoding="latin1")
df.head()
```

```
Out[2]:
```

Row ID	Order ID	Order Date	Ship Date	Product Line
1	2	21 11 21	11 11 21	Se
CA-	-	/ /	/ /	(
CA-	-	/ /	/ /	Se
CA-	-	/ /	/ /	Se
US-	-	/ /	/ /	Sta
US-	-	/ /	/ /	Sta

rows × columns

Step 3 : Load Dataset into SQL Database

```
In [3]: con = duckdb.connect()
con.register("superstore", df)
```

```
Out[3]: <_duckdb.DuckDBPyConnection at 0x2b48c2bb030>
```

Step : Explore the Table

```
In [4]: con.sql("""
SELECT *
FROM superstore
LIMIT 5
""").df()
```

Out[4]:

Row ID	Order ID	Order Date	Ship Date	
1	CA-211111111	2012-01-01	2012-01-01	Se
2	CA-211111111	2012-01-01	2012-01-01	Se
3	CA-211111111	2012-01-01	2012-01-01	Se
4	US-211111111	2012-01-01	2012-01-01	Sta
5	US-211111111	2012-01-01	2012-01-01	Sta

rows × columns

Total Rows

```
In [5]: con.sql("""
SELECT COUNT(*) AS Total_Rows
FROM superstore
""").df()
```

Out[5]:

Total_Rows

Column Names

```
In [6]: df.columns
```

```
Out[6]: Index(['Row ID', 'Order ID', 'Order Date', 'Ship Date', 'Ship Mode',  
              'Customer ID', 'Customer Name', 'Segment', 'Country', 'City', 'State',  
              'Postal Code', 'Region', 'Product ID', 'Category', 'Sub-Category',  
              'Product Name', 'Sales', 'Quantity', 'Discount', 'Profit'],  
             dtype='object')
```

Data Types

```
In [7]: df.dtypes
```

```
Out[7]: Row ID          int64  
Order ID          object  
Order Date        object  
Ship Date         object  
Ship Mode         object  
Customer ID       object  
Customer Name     object  
Segment          object  
Country           object  
City              object  
State             object  
Postal Code       int64  
Region            object  
Product ID        object  
Category          object  
Sub-Category      object  
Product Name      object  
Sales             float64  
Quantity          int64  
Discount          float64  
Profit            float64  
dtype: object
```

Apply WHERE Filters

```
In [8]: #Region = West  
con.sql("""  
SELECT *  
FROM superstore  
WHERE Region='West'  
""").df()
```

Out[8]:

rows ×

columns

```
In [9]: #Category = Furniture
con.sql("""
SELECT *
FROM superstore
WHERE Category='Furniture'
""").df()
```

Out[9]:

rows × columns

```
In [ ]: #Sales > 500
        con.sql("""
        SELECT *
```

```
FROM superstore
WHERE Sales>500
""").df()
```

GROUP BY Queries

```
In [10]: con.sql("""
SELECT Category,
SUM(Sales) AS Total_Sales
FROM superstore
GROUP BY Category
""").df()
```

```
Out[10]:
```

Category	Total_Sales
Furniture	.
Technology	.
Office Supplies	.

```
In [11]: #Quantity by Region
con.sql("""
SELECT Region,
SUM(Quantity) AS Total_Quantity
FROM superstore
GROUP BY Region
""").df()
```

```
Out[11]:
```

Region	Total_Quantity
Central	.
East	.
South	.
West	.

```
In [12]: #Average Profit
con.sql("""
SELECT Category,
AVG(Profit) AS Average_Profit
FROM superstore
GROUP BY Category
""").df()
```

```
Out[12]:
```

Category	Average_Profit
Technology	.
Furniture	.
Office Supplies	.

ORDER BY and LIMIT

```
In [13]:
```

```
con.sql("""
SELECT "Product Name",
SUM(Sales) AS Total_Sales
FROM superstore
GROUP BY "Product Name"
ORDER BY Total_Sales DESC
LIMIT 10
""").df()
```

Out[13]:

	Product Name	Total_Sales
	Canon imageCLASS Advanced Copier	.
	Fellowes PB Electric Punch Plastic Comb Bin...	.
	Cisco TelePresence System EX Videoconferenci...	.
	HON Series Task Chairs for Big and Tall	.
	GBC DocuBind TL Electric Binding System	.
	GBC Ibimaster Manual ProClick Binding System	.
	Hewlett Packard LaserJet Copier	.
	HP Designjet T Inkjet Large Format Printer ...	.
	GBC DocuBind P Electric Binding System	.
	High Speed Automatic Electric Letter Opener	.

In [14]:

```
#Top Categories
con.sql("""
SELECT Category,
SUM(Sales) AS Total_Sales
FROM superstore
GROUP BY Category
ORDER BY Total_Sales DESC
""").df()
```

Out[14]:

Category	Total_Sales
Technology	.
Furniture	.
Office Supplies	.

Business Queries

In [16]:

```
#Monthly Sales Trend
con.sql("""
SELECT strftime('%Y-%m', CAST("Order Date" AS DATE)) AS Month,
SUM(Sales) AS Total_Sales
FROM superstore
GROUP BY Month
ORDER BY Month
""").df()
```



```

-----
ConversionException                                Traceback (most recent call last)
Cell In[16], line 9
      1 #Monthly Sales Trend
      3 con.sql(
      4
      5
      6
      7
      8
----> 9 ).df()

ConversionException: Conversion Error: invalid date field format: "11/8/2016"
expected format is (YYYY-MM-DD) when casting from source column Order Date

LINE 1: SELECT strftime('%Y-%m', CAST("Order Date" AS "DATE")) AS "Month",
sum(Sales) AS Tota...
^

```

```

In [17]: #Monthly Sales Trend

con.sql("""
SELECT strftime('%Y-%m', strftime("Order Date", '%m/%d/%Y')) AS Month,
SUM(Sales) AS Total_Sales
FROM superstore
GROUP BY Month
ORDER BY Month
""").df()

```

21 02

0000 1 0000

Total_Sales


```
In [19]: #Duplicate Records
con.sql("""
SELECT *,
COUNT(*) AS Duplicate_Count
FROM superstore
GROUP BY ALL
HAVING COUNT(*)>1
""").df()
```

```
Out[19]:
```

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	...	Re
rows × columns											

Data Validation

```
In [20]: con.sql("""
SELECT COUNT(*) AS Total_Rows
FROM superstore
""").df()
```

```
Out[20]:
```

Total_Rows

```
In [21]: #Missing Values
df.isnull().sum()
```

```
Out[21]:
```

Row ID	0
Order ID	0
Order Date	0
Ship Date	0
Ship Mode	0
Customer ID	0
Customer Name	0
Segment	0
Country	0
City	0
State	0
Postal Code	0
Region	0
Product ID	0
Category	0
Sub-Category	0
Product Name	0
Sales	0
Quantity	0
Discount	0
Profit	0
dtype:	int64

```
In [22]: #Duplicate Rows
df.duplicated().sum()
```

```
Out[22]: np.int64(0)
```

Step : Brief Insights

In []: *# Brief Insights*

- Technology generated the highest sales.
- The West region contributed significantly to total sales.
- A few customers accounted **for** a large share of revenue.
- The dataset contained very few missing values.
- No major duplicate records were found.

In []: